머리 탐지 학습 결과 보고서

군중 밀집도 기반 인파 사고 조기 경보 · 추가 기능 Y① | 202104183 김현중, 202204201 김태섭 | 2026-11-12

1. 한눈에 보기

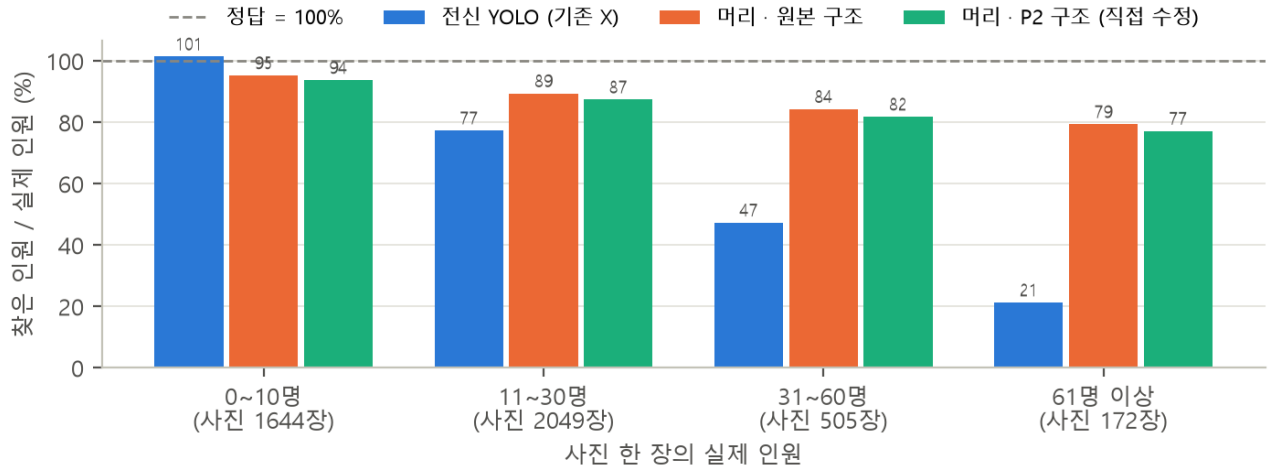
기존 YOLO는 사람 **전신**을 찾는다. 우리는 같은 YOLO 학습 코드를 이해하고 **데이터·출력 클래스·모델 구조**를 바꿔 **머리**를 찾도록 직접 학습시켰다. CrowdHuman 검증 사진 4,370장(실제 인원 85,093명)에서 인원을 세어 비교했다.

모델	머리 mAP50	머리 mAP50-95	인원 오차 (MAE, 명/장)	찾은 비율 (전체)	찾은 비율 (61명 이상 사진)	속도 (ms/장)
전신 YOLO (기존 X)	-	-	8.15	62.0%	21.3%	4.9
머리 · 원본 구조	0.812	0.504	3.10	87.0%	79.4%	4.8
머리 · P2 구조 (직접 수정)	0.828	0.516	3.40	84.9%	77.2%	6.8

mAP: 머리 박스 위치까지 맞혔는지(검증 4,370장, IoU 기준). 인원 오차: 사진마다 |센 인원 - 실제 인원|의 평균. 찾은 비율: 센 인원 합 ÷ 실제 인원 합(100%에 가까울수록 좋음). 속도: RTX 4070, 입력 960, 추론만(검증 300장 × 3회, 세 모델 번갈아 측정한 중앙값).

핵심 결과

- 인원 오차(MAE): 전신 8.1명 → 머리 3.1명 (**62% 감소**, 머리 · 원본 구조).
- 61명 이상 붐비는 사진: 전신은 실제 인원의 21.3%만 셌지만, 머리는 79.4%를 셌다. **붐빌수록 머리가 필요하다**는 근거.
- 직접 수정한 P2 구조: mAP50 0.812 → 0.828, mAP50-95 0.504 → 0.516로 **박스 정확도는 올랐지만**, 인원 세기(MAE 3.10 → 3.40)와 16px 미만 머리 찾기는 **나아지지 않았고** 속도는 4.8 → 6.8 ms/장으로 느려졌다 (4-2 참고).



검증 사진을 실제 인원으로 묶어, 각 모델이 센 인원 ÷ 실제 인원을 비교했다. 100%보다 낮으면 덜 센 것.

2. 이전 코드와 무엇이 달라졌나

	이전 (기존 기능 X)	현재 (추가 기능 Y①)
찾는 대상	사람 전신	사람 머리 (모자·뒤통수 포함)
학습 데이터	COCO (Ultralytics가 학습해 배포)	CrowdHuman 15,000장 · 머리 287,106개 (직접 변환)
출력 클래스	COCO 80종 중 person 사용	head 1종 (Detect 층 출력 80 → 1)
모델 구조	YOLO11n 원본 (P3·P4·P5에서 탐지)	① base: 원본 그대로 ② P2: 작은 머리용 P2 탐지층 추가 (직접 수정)
시작 가중치	COCO 학습 결과	COCO 가중치에서 전이학습 (맞는 층만 이어받음)
코드	src/yolo_person.py	src/convert_crowdhuman.py · models/head_p2.yaml · src/train_head.py · src/eval_heads.py

Ultralytics 라이브러리 코드는 고치지 않았다. 바꾼 것은 **무엇을 가르치는지(데이터·클래스)**와 **모델 구조 설정 파일**이다.

2-1. 데이터 변환: CrowdHuman → YOLO 머리 라벨

CrowdHuman 라벨(.odgt)은 사진 한 장마다 사람 목록이 있고, 사람마다 전신·보이는 부분·머리 박스가 있다. 우리는 머리 박스(hbox)만 골라 YOLO 형식 "0 cx cy w h"(0~1 비율)로 바꿨다.

- **mask 영역**(포스터·마네킹, 또는 너무 빽빽해 라벨을 못 단 군중)과 **ignore 표시된 머리**는 학습에서 뺐다.
- 그냥 빼면 그 안의 진짜 머리가 '머리 아님'으로 학습된다. Ultralytics에는 무시 영역 기능이 없으므로 **그 영역을 회색(114)으로 칠해 사진에서 지웠다**. (학습 105,490곳, 검증 30,000곳)
- 사진 밖으로 나간 박스는 경계에 맞춰 자르고, 2픽셀보다 작으면 뺐다.
- 통계: 머리의 약 30%가 높이 16픽셀 미만 → 작은 머리를 잘 찾는 구조가 필요하다는 근거.

2-2. 모델 구조 변경: P2 탐지층 추가 (models/head_p2.yaml)

원본 YOLO11n은 1/8·1/16·1/32 해상도(P3·P4·P5) 세 곳에서 탐지한다. 16픽셀 머리는 P3 특징 지도에서 2칸뿐이라 놓치기 쉽다. 그래서 백본 2번 층의 **1/4 해상도(P2) 특징을 끌어와** 한 단계 더 촘촘한 곳에서도 탐지하게 했다. Ultralytics에는 YOLO11용 P2 설정이 없어(YOLOv8용만 있음) 원본 설정을 직접 고쳤다.

원본 yolo11.yaml (head 부분)

```
- [-1, 1, nn.Upsample, [None, 2, "n"]]
- [[-1, 6], 1, Concat, [1]]
- [-1, 2, C3k2, [512, False]]
- [-1, 1, nn.Upsample, [None, 2, "n"]]
- [[-1, 4], 1, Concat, [1]]
- [-1, 2, C3k2, [256, False]]
- [-1, 1, Conv, [256, 3, 2]]
- [[-1, 13], 1, Concat, [1]]
- [-1, 2, C3k2, [512, False]]
- [-1, 1, Conv, [512, 3, 2]]
- [[-1, 10], 1, Concat, [1]]
- [-1, 2, C3k2, [1024, True]]
- [[16, 19, 22], 1, Detect, [nc]]
```

직접 수정한 head_p2.yaml (head 부분, '+ 추가' 표시)

```
  # 위에서 아래로 (큰 특징 → 작은 특징), 원본과 동일
- [-1, 1, nn.Upsample, [None, 2, "n"]] # 11
- [[-1, 6], 1, Concat, [1]] # 12 cat backbone P4
- [-1, 2, C3k2, [512, False]] # 13
- [-1, 1, nn.Upsample, [None, 2, "n"]] # 14
- [[-1, 4], 1, Concat, [1]] # 15 cat backbone P3
- [-1, 2, C3k2, [256, False]] # 16 (P3/8)
  # + 추가: P2까지 한 단계 더 내려감
- [-1, 1, nn.Upsample, [None, 2, "n"]] # 17
- [[-1, 2], 1, Concat, [1]] # 18 cat backbone P2
- [-1, 2, C3k2, [128, False]] # 19 (P2/4-xsmall) ← 새 탐지 위치
  # 아래에서 위로 다시 올라가며 합침 (P2 → P3 → P4 → P5)
- [-1, 1, Conv, [128, 3, 2]] # 20 + 추가
- [[-1, 16], 1, Concat, [1]] # 21 + 추가: cat P3
- [-1, 2, C3k2, [256, False]] # 22 (P3/8-small)
- [-1, 1, Conv, [256, 3, 2]] # 23
- [[-1, 13], 1, Concat, [1]] # 24 cat P4
- [-1, 2, C3k2, [512, False]] # 25 (P4/16-medium)
- [-1, 1, Conv, [512, 3, 2]] # 26
- [[-1, 10], 1, Concat, [1]] # 27 cat P5
- [-1, 2, C3k2, [1024, True]] # 28 (P5/32-large)
- [[19, 22, 25, 28], 1, Detect, [nc]] # Detect(P2, P3, P4, P5)
```

	원본 구조 (base)	P2 구조
탐지 위치(stride)	8 · 16 · 32	4 · 8 · 16 · 32
파라미터 수	2,604,883	2,666,820
연산량 (GFLOPs, 640 기준)	6.6	10.5
COCO에서 이어받은 가중치	493/499	297/593

P2 구조는 파라미터는 거의 같지만, 큰 해상도에서 계산하므로 연산량이 늘어난다.

3. 어떻게 학습했나

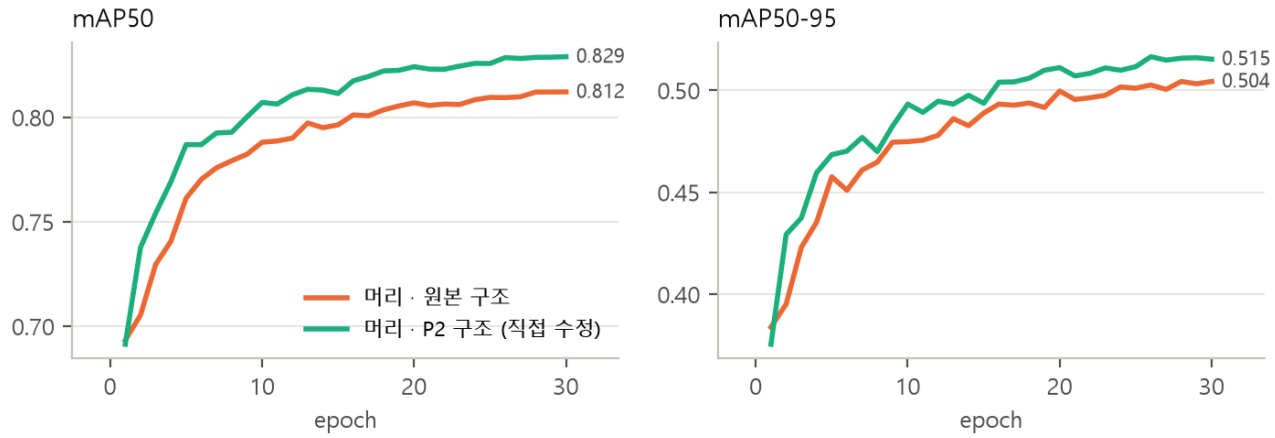
항목	값	이유
데이터	학습 15,000장 / 검증 4,370장 (CrowdHuman 공식 분할)	검증 사진은 학습에 쓰지 않음
입력 크기	960	머리가 작아 기본값 640보다 크게
epoch / batch	30 / base 16 · P2 8	P2는 batch 16에서 GPU 메모리(12GB)가 모자라 8로 줄임. Ultralytics는 기울기를 모아 유효 batch 64로 맞추므로(16×4 = 8×8) 학습률·최적화 조건은 같음
최적화	AdamW (lr 0.002)	Ultralytics 자동 선택
장비	RTX 4070 (12GB), Python 3.13, torch 2.15 + CUDA	Colab 대신 이 PC GPU
학습 시간	base 1.3시간 · P2 5.7시간	P2는 22 epoch에서 한 번 멈췄다가 resume으로 이어서 학습 (밤에는 다른 작업과 GPU를 나눠 써서 더 느렸음)
가장 좋은 epoch	base 30 · P2 26	검증 점수가 가장 높은 가중치를 best.pt로 저장

학습 명령 (같은 Ultralytics 학습 코드 model.train)

```
# base: COCO 가중치를 불러오면 Detect 층 출력만 1클래스로 새로 만들어짐
YOLO('yolo11n.pt').train(data='head.yaml', epochs=30, imgsz=960, batch=16)

# P2: 직접 수정한 구조를 만든 뒤, 모양이 맞는 층만 COCO 가중치를 이식
YOLO('head_p2.yaml').load('yolo11n.pt').train(data='head.yaml', epochs=30, imgsz=960, batch=8)
```

학습 곡선 (검증 사진 점수, epoch마다)



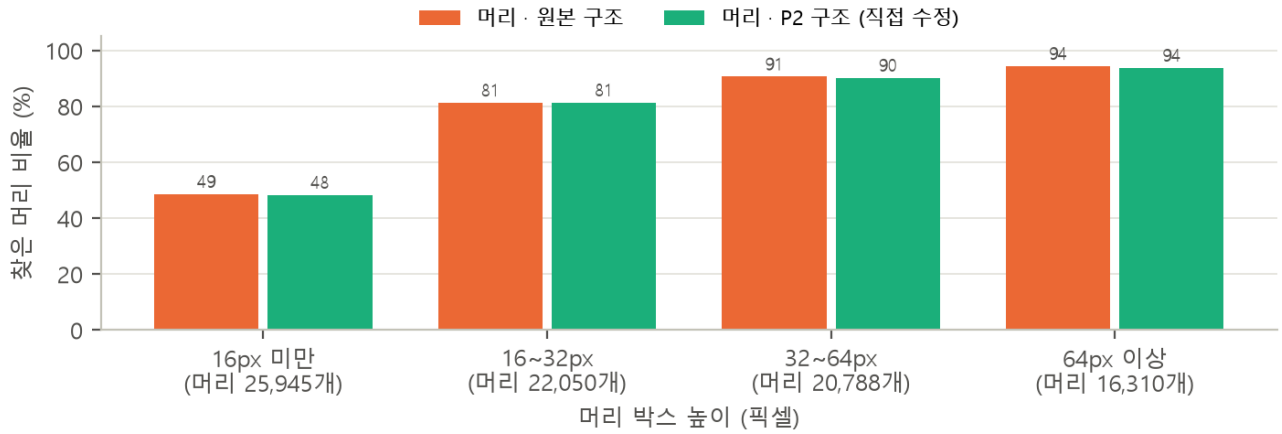
선이 오르다 평평해지면 더 배울 것이 적다는 뜻. 끝의 숫자는 마지막 epoch 점수.

4. 결과

4-1. 사람이 많을수록 전신 YOLO는 덜 센다 (1쪽 그림)

- 전신 YOLO: 101.4% → 77.3% → 47.3% → 21.3% (사람이 많아질수록 몸이 가려져 박스를 놓치고, 겹친 박스는 하나로 합쳐짐).
- 머리 · 원본 구조: 95.3% → 89.4% → 84.3% → 79.4% (머리는 가려지는 일이 적어 붐벼도 비율이 유지됨).
- 머리 · P2 구조 (직접 수정): 93.8% → 87.5% → 81.8% → 77.2% (머리는 가려지는 일이 적어 붐벼도 비율이 유지됨).

4-2. 머리 크기별로 찾은 비율 (P2 구조의 효과)



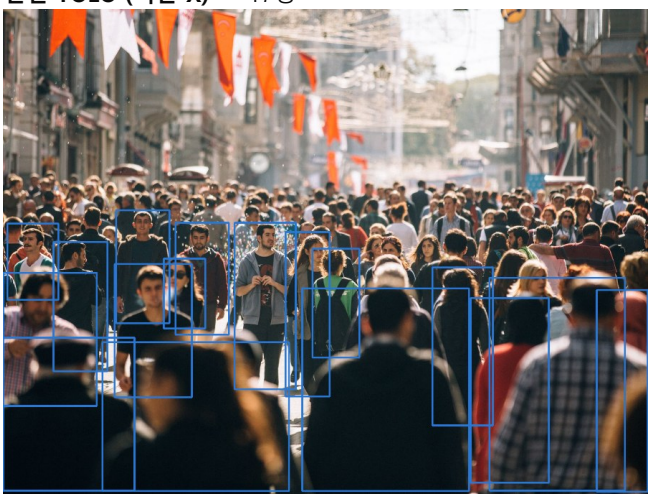
정답 머리와 IoU 0.5 이상 겹치는 예측이 있으면 '찾음'. 작은 머리에서 두 구조의 차이를 본다.

- 16px 미만: base 48.7% → P2 48.4% (-0.3%p)
- 16~32px: base 81.3% → P2 81.4% (+0.1%p)
- 32~64px: base 90.7% → P2 90.2% (-0.4%p)
- 64px 이상: base 94.4% → P2 93.8% (-0.6%p)
- **해석**: 모든 크기에서 차이가 1%p 이내다. P2를 넣어도 16px 미만 머리는 여전히 절반 정도만 찾았다. P2의 mAP 상승(mAP50-95 +1.2%p)은 '더 많이 찾기'가 아니라 **박스 위치를 더 정확히 맞춘 것**에서 나왔다(mAP50-95는 IoU 0.5~0.95를 평균해 위치 정확도를 더 많이 반영).
- **작은 머리가 안 잡히는 이유(추정)**: 입력 960으로 줄이면서 원본에서 이미 작던 머리가 몇 픽셀까지 작아진다. 탐지층을 늘려도 입력에 정보가 없으면 찾을 수 없다. 다음 실험은 입력 크기 키우기, 사진을 조각내 확대해서 탐지하기(타일링), 모델별 신뢰도 기준 맞추기.

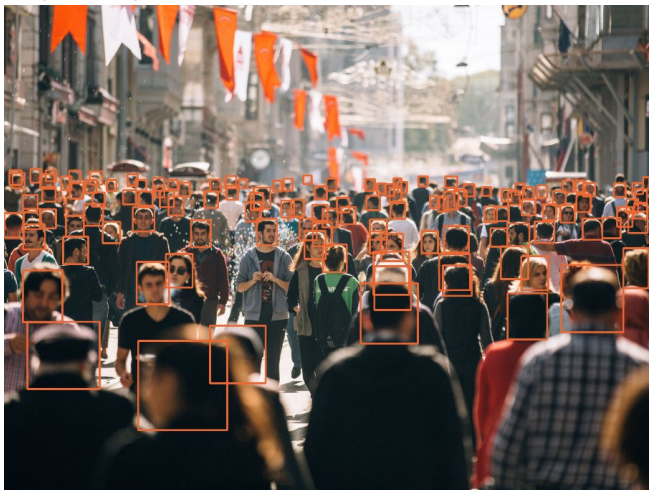
4-3. 같은 사진, 모델별 결과

DM-Count 예시 사진(학습에 쓰지 않은 사진). 박스 색은 모델 색과 같다.

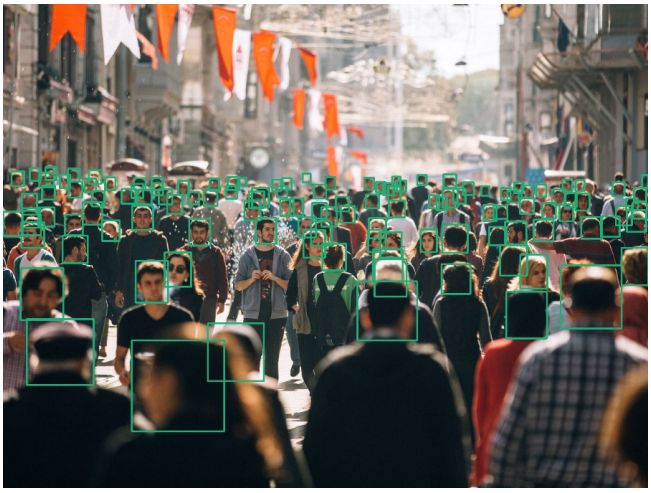
전신 YOLO (기존 X) — 17명



머리 · 원본 구조 — 123명



머리 · P2 구조 (직접 수정) — 122명



5. 정리와 발표 포인트

- **기존 기능을 다르게 적용**: 같은 YOLO 학습 코드에 데이터·클래스·구조를 바꿔 '전신' 대신 '머리'를 찾게 했다 (공지 예시의 '마스크 → 썬글라스 추가'와 같은 방식).
- **직접 이해하고 바꾼 부분**: ① CrowdHuman 라벨 변환 규칙(무시 영역 회색 처리) ② Detect 층 80 → 1 클래스와 전이학습 ③ YOLO11 구조 파일에 P2 탐지층 추가.
- **왜 머리인가**: 붐비는 사진(61명 이상)에서 전신은 21.3%, 머리는 79.4%를 셌다. 인파 사고는 붐빌 때 일어나므로 이 차이가 핵심.
- **구조 수정의 효과**: P2 구조는 mAP가 올랐지만(박스 위치 정확도), 인원 세기와 작은 머리 찾기는 그대로였고 속도는 느려졌다. 즉 **작은 머리 문제는 탐지층 추가만으로는 풀리지 않았다**. 가설을 세우고 직접 확인해 '안 됐다'까지 보여 주는 것도 실험 결과이고, 인원 세기에는 원본 구조(head_base)를 쓴다.

한계

- 평가는 CrowdHuman 검증 사진 기준이다. CCTV처럼 위에서 내려다본 사진에서는 따로 확인해야 한다.
- 인원은 신뢰도 0.25 이상 박스 개수로 셌다. 기준값에 따라 더 세거나 덜 셀 수 있다.
- P2는 메모리 때문에 batch 8로 학습했다(base는 16). 유효 batch는 64로 같지만 BatchNorm 통계는 작은 batch로 계산된다.
- 정답 인원은 무시 영역을 뺀 머리 라벨 개수다. 회색으로 지운 영역의 사람은 정답에도, 예측에도 들어가지 않는다.

다음 할 일

- 머리 모델(head_base)을 crowd_final.py에 넣어 P2PNet(임시 대체)을 바꾸기
- 작은 머리(16px 미만, 전체의 30%) 찾기: 입력 크기 키우기 · 사진 조각내 확대 탐지(타일링) 비교
- 전신·머리 결합 세기 (머리 중심이 전신 박스 위쪽 1/3 안이면 같은 사람)
- 바닥 4점 원근 변환으로 ㎡당 밀도 → 경보 (응용)